

CLAUDE CODE

Turn your best prompt into a /command

One markdown file. Your whole team types /commit and gets it right every time.

THE GAP

You're re-typing your best prompts

The good prompt lives in
someone's head or a stale doc.
Ship it in the repo instead, where
everyone runs the same thing.

THE RULE

Filename = command

Drop a .md file in
.claude/commands/ and commit
it.

- `deploy.md -> /deploy`
- Project (shared): `.claude/commands/`
- Personal (you): `~/.claude/commands/`

THE FILE

A real /commit command

Frontmatter pre-approves the tools. The body is the prompt.

- ---
- allowed-tools: Bash(git add *), Bash(git commit *)
- description: Create a git commit
- ---

THE MAGIC

The ! prefix inlines bash output

It runs the command and pastes the OUTPUT into the prompt before Claude reads it.

- - Status: !`git status`
- - Diff: !`git diff HEAD`
- Claude writes from the real diff, not a guess

ARGUMENTS

Pass input straight in

`$ARGUMENTS` is everything typed after the command.

- `/fix-issue 123 -> $ARGUMENTS = 123`
- `$0` = first arg, `$1` = second (0-based!)
- argument-hint: [issue-number]

FILE REFS

Pull files in with @

Inline a file's contents into the prompt, always current.

- @package.json
- @tsconfig.json
- No copy-paste, never stale

NEW IN 2026

Commands merged into skills

Same machinery now, two paths to the same command.

- `.claude/commands/deploy.md`
- `= .claude/skills/deploy/SKILL.md`
- Both make `/deploy`. Old files still work.

CHEAT SHEET

Build one in 60 seconds

- 1. echo prompt > .claude/commands/x.md
- 2. add !`bash` for live context
- 3. add \$ARGUMENTS for input
- 4. git commit -> team has /x



YOUR MOVE

Ship one command this week

We build command libraries for
engineering teams. Comment
SETUP and I'll share ours.

Follow Noblerr →